

Interpreter python, podstawowe operacje (zmienne, warunku, pętle, funkcje)

Spis treści

Spis treści

Interpreter / uruchamianie kodu

Uruchamianie interpretera i sprawdzenie jego wersji

Uruchamianie plików pythonowych

Podstawowe polecenia

Printowanie

Komentarze w kodzie

Zmienne

Warunki

Pętle

Funkcje

Zadania do wykonania

Interpreter / uruchamianie kodu

Uruchamianie interpretera i sprawdzenie jego wersji

```
python --version
```

Uruchamianie plików pythonowych

```
python helloworld.py
```

Gdzie `helloworld.py` jest nazwą pliku uruchamianego.

Podstawowe polecenia

Printowanie

1. Zwykłe hello world `print("Hello, World!")`

Output:

```
Hello, World!
```

2. Print z konkatencją stringów

```
name = 'Martin'

print('Hello, ' + name + '! How are you?')
print(f'Hello, {name}! How are you?')
print('Hello, {}! How are you?'.format(name))
```

Output:

```
Hello, Martin! How are you?
Hello, Martin! How are you?
Hello, Martin! How are you?
```

3. Print wielu zmiennych

```
print('Martin', 'Tom', 'Eva')
print('Martin', 'Tom', 'Eva', sep=';')
```

Output:

```
Martin Tom Eva
Martin;Tom;Eva
```

Komentarze w kodzie

1. W osobnej lini

```
# My comment  
print("Hello, World!")
```

2. W tej samej lini co kod

```
print("Hello, World!") # My comment
```

3. Wielolinijkowe komentarze

```
'''  
My comment - first line  
My comment - second line  
My comment - third line  
'''  
print("Hello, World!")
```

Zmienne

1. Przypisywanie wartości

```
x = 11  
y = "Martin"  
  
print(x, type(x))  
print(y, type(y))
```

Output:

```
11 <class 'int'>  
Martin <class 'str'>
```

2. Castowanie typów

```
x = str(11)
y = int(11)
z = float(11)

print(x, type(x))
print(y, type(y))
print(z, type(y))
```

Output:

```
11 <class 'str'>
11 <class 'int'>
11.0 <class 'float'>
```

3. Typy zmiennych

W przystępny sposób zebrane i opisane na [w3schools](https://www.w3schools.com/python/python_variables.asp)

Warunki

1. Najprostszy `if`

```
x = 12

if x > 0:
    print('x greater than zero')
```

Output:

```
x greater than zero
```

2. `if` + `else`

```
x = -11

if x >= 0:
    print('x greater than zero')
else:
    print('x lower than zero')
```

Output:

```
x lower than zero
```

3. `if` + `elif`

```
x = -11

if x >= 0:
    print('x greater than zero')
elif x > -20:
    print('x lower than zero and greater than -20')
```

Output:

```
x lower than zero and greater than -20
```

Pętle

1. `while`

```
x = 0

while x < 3:
    print(x)
    x += 1
```

Output:

```
0
1
2
```

2. `while` + `else`

```
x = 0

while x < 3:
    print(x)
    x += 1
else:
    print('end loop')
```

Output:

```
0
1
2
end loop
```

3. `for`

```
names = ['Martin', 'Tom', 'Eva']

for name in names:
    print(name)
```

4. `for` + `range`

```
for i in range(3):
    print(i)
```

Output:

```
0
1
2
```

5. `for` + `else`

```
for i in range(3):
    print(i)
else:
    print('end loop')
```

Output

```
0
1
2
end loop
```

6. `break` w celu stopowania pętli (działa dla `while` jak i `for`)

```
for name in names:
    if name == 'Tom':
        break
    print(name)
```

Output

```
Martin
```

7. `continue` w celu pomijania kroku pętli (działa dla `while` jak i `for`)

```
for name in names:
    if name == 'Martin':
```

```
        continue
    print(name)
```

Output:

```
Tom
Eva
```

8. Listy/słowniki składane częstą alternatywą dla pętli

Przykładowy problem do rozwiązania: Przeiteruj listę imion i zapisz do nowej listy długość każdego z nich.

1. Rozwiązanie z wykorzystaniem pętli `for` :

```
names = ['Martin', 'Tom', 'Eva']
name_lengths = []
for name in names:
    name_lengths.append(len(name))

print(name_lengths)
```

2. Rozwiązanie dla listy składanej:

```
names = ['Martin', 'Tom', 'Eva']
name_lengths = [len(name) for name in names]

print(name_lengths)
```

Oba rozwiązania dają ten sam output (listę długości imion):

```
[6, 3, 3]
```

Funkcje

1. Definicja najprostrzej funkcji + jej wywołanie


```
def my_awesome_function():  
    print('print from function')  
  
my_awesome_function()
```

Output:

```
print from function
```

2. Funkcja z 2 parametrami

```
def print_name(name, surname):  
    print(f'Hi {name}, {surname}')
```



```
print_name('Martin', 'Williams')
```

Output:

```
Hi Martin, Williams
```

3. Funkcja z parametrami o domyślnych wartościach

```
def print_name(name='John', surname='Williams'):  
    print(f'Hi {name}, {surname}')
```



```
print_name('Martin')
```

Output:

```
Hi Martin, Williams
```

4. Funkcja zwracająca wynik

```
# function definition
def add_numbers(x, y):
    return x + y

# assign function result to variable
sum_two_numbers = add_numbers(1, 2)

# print variable
print(sum_two_numbers)
```

Output:

3

Zadania do wykonania

1. Dla początkujących, którzy mają problemy ze zrozumieniem powyższego materiału:
 - a. Szybkie 95 zadań na stronie w3schools
<https://www.w3schools.com/python/exercise.asp> - potwierdzenie wykonania w formie screena umieszczonego w repozytorium.
2. Dla osób posiadających podstawową wiedzę o pythonie:
 - a. Utwórz funkcję, która **otrzyma w parametrze** listę 5 imion, a następnie wyświetli każde z nich.
 - b. Utwórz funkcję, która **otrzyma w parametrze** listę zawierającą 5 dowolnych liczb, każdy jej element pomnoży przez 2, a na końcu ją **zwróci**. Zadanie należy wykonać w 2 wersjach:
 - i. Wykorzystując pętlę `for`.
 - ii. Wykorzystując listę składaną.
 - c. Utwórz funkcję, która **otrzyma w parametrze** listę 10 liczb (rekomendowane wykorzystanie funkcji `range`), a następnie wyświetli

jedynie parzyste elementy.

- d. Utwórz funkcję, która **otrzyma w parametrze** listę 10 liczb (rekomendowane wykorzystanie funkcji `range`), a następnie wyświetli co drugi element.

Wykonane zadania należy umieścić w repozytorium na gałęzi `python_basic`

Każde zadanie należy rozbijać na osobne pliki lub jeśli ktoś potrzebuje to na osobne katalogi z zachowaniem nazewnictwa `zad_1` , `zad_2` ... `zad_n` .